Strata — Architecture

Knowledge Base as Code

strata-kb v1.34.0 · 2026-11-01

1. Purpose and scope

Strata turns large reference documents into a knowledge base that humans review in a pull request and AI agents query for exactly the passage they need.

This document describes **how the system is built**: its data model, its components, the guarantees each one provides, and the topologies it is deployed in. It is written for architects, platform engineers and technical leads evaluating or operating Strata. Day-to-day instructions live in the three role guides that accompany it.

Strata is domain-agnostic. Nothing in the design encodes an industry, a document family or a language. The unit of work is "a long document whose details matter"; what that document is about never reaches the code.

1.1 Design constraints

Four constraints shaped every decision below.

Constraint	Consequence in the design
Detail must survive summarization	Tables are extracted by code, never rewritten by a model, and diffed automatically on every build
Review must be possible without special tooling	The store is plain YAML and Markdown in Git; the review surface is an ordinary pull request
Lookup must be cheap enough to run on every question	Four layers, so a query loads a slice rather than a corpus; no model call happens during a lookup
Knowledge must be shared, not duplicated	A hub holds one copy of each document; other repos reference it rather than re-ingesting it

1.2 What Strata is not

- **Not a database.** There is no server holding state. Every artefact is a file under version control, and every query reads a working copy of a Git tree.
- **Not a RAG pipeline with a vector store as the source of truth.** The search index is derived and disposable; the Markdown is authoritative and can rebuild the index at any time.
- **Not an autonomous documentation system.** A model drafts prose. A human always signs off before content becomes reachable.

2. The data model

2.1 Four layers

A Strata store keeps every document at four levels of detail simultaneously.

Layer	Artefact	Content	Written by
L0	`.kb/index.yaml`	One entry per document: id, revision, tags, a ≤ 30-word description	`kb ingest`, refined by a model
L1	`.kb/<doc>/_manifest.yaml`	One entry per section: id, title, a ≤ 25-word summary, status, token counts, content hashes	`kb ingest` scaffolds, `kb summarize` fills
L2	`.kb/<doc>/<part>.md`	Condensed prose in the source's own language, with every table verbatim	`kb summarize` (prose only) and code (tables)
L3	`.kb/<doc>/<part>.raw.md`	The full extracted original, nothing removed	`kb ingest`, never a model

The layers are not alternatives; they are a descent. A query walks L0 to choose documents, L1 to choose sections, and loads L2 for the few that matter. L3 is opened only when a caller needs the unabridged original.

```
 L0  index.yaml          ~200 tokens      "which documents exist"
  |
L1  _manifest.yaml      ~10^4 tokens     "which sections exist"
  |
L2  <part>.md           ~10^5 tokens     "what a section says"
  |
L3  <part>.raw.md       ~10^5 tokens     "exactly what the source said"
```

The cost model follows directly: a lookup that needs three sections pays for L0, a tag-filtered slice of L1, and three L2 sections — not for the document.

2.2 Sections

A section is the atom of citation and retrieval. It has:

- **A stable id** derived from the source's own numbering (`5.129`, `ch2`, `appendix-3`). The id is the citation key, so it is never reassigned; a renumbered section is modelled as one section removed and another added.
- **A status**: `pending` (no summary yet) → `summarized` (a model wrote one) → `reviewed` (a human approved it, recorded with who and when).
- **Two content hashes**: `l3_sha256` fixes the original the summary was written against, and `reviewed.l2_sha256` fixes the summary a human approved. Either changing out from under the other is a build failure, not a silent drift.

2.3 The table invariant

This is the load-bearing guarantee of the whole system.

> Prose may be condensed by a model. **Tables may not.** Every table is extracted from the source by code, written identically into L2 and L3, and compared on every build.

`kb build` normalises only presentation — non-breaking spaces, tabs, trailing whitespace, column alignment markers, redundant separator rows — and then compares byte for byte. The comparison runs in both directions, so a table that was altered, dropped, duplicated, reordered or invented fails the build.

The practical effect is that the most error-prone content in a technical document is the content a model is structurally unable to corrupt.

2.4 Assets

Images are extracted during ingestion and stored content-addressed as `.kb/<doc>/assets/<sha256>.png` or `.webp`. Identical bytes always produce the same filename, so re-ingesting a document never duplicates an asset and a figure reused across chapters is stored once.

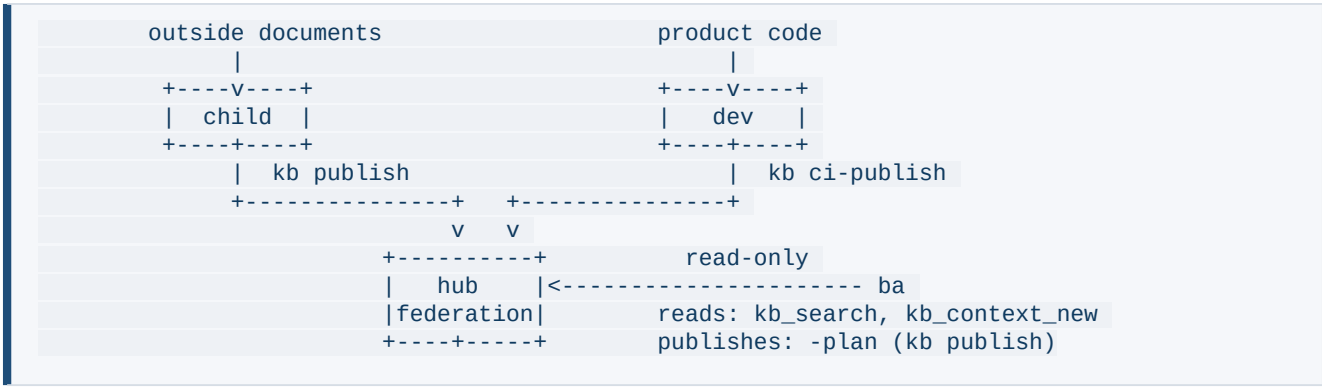
Descriptions are never generated. They are taken from the source — the image's own caption where one exists, falling back to OCR of text baked into the image. L3 carries a relative Markdown image reference; L2 carries the same description as a `Figure: …` line so the image participates in keyword and semantic search.

---

3. Repository kinds

Strata is not one repository. It is a small set of repository roles that together form a knowledge supply chain.

Kind	Produces	Consumes	Publishes to the hub
`hub`	`federation/`, the search service	every child's publish	itself, and optionally an upstream hub
`child`	domain documents from outside sources	—	yes
`ba`	tickets and mission plans	the hub, read-only	yes (`-plan` only)
`dev`	product code, plus knowledge about that code	the hub, read-only	yes (`-code`, `-svc`)



```
                              |
                  kb query / MCP / Web UI
```

`kb init --kind <kind>` scaffolds a repository for its role and records the choice in `.kb/config.yaml`. The kind is not cosmetic: it determines which commands are available, which slash commands and CI workflows are generated, and how the MCP client is wired (stdio on a hub, HTTP everywhere else).

3.1 Why the roles are separate

A `child` ingests documents from **outside** the repository. A `dev` repository ingests **nothing** and authors only knowledge about its own source code. Keeping these apart means a product repository can publish knowledge continuously from CI without ever acquiring the ability to add arbitrary documents to the corpus.

A `ba` repository is read-only against the *knowledge* base. It cites, it never contributes, and it never ingests or summarizes. That keeps requirements traceable to a knowledge version without giving the requirements process a way to change the knowledge. The one thing it publishes is its own plan, a generated document (`<repo_id>-plan`) listing only the missions and tickets the analyst has marked ready (§8). The plan sits beside the knowledge; it does not edit it.

---

4. Components

4.1 The `kb` CLI

One executable, grouped by lifecycle stage.

Group	Commands
Scaffolding	`init`, `docker-setup`
Authoring	`ingest`, `summarize`, `status`, `build`, `approve`
Retrieval	`query`, `get`, `stats`, `tags`
Federation	`publish`, `ci-publish`, `reindex`, `doctor`, `assets`
Citations	`context new`, `resolve`, `diff`
Code knowledge	`code-ingest`, `svc note`
Gates	`ticket lint`, `mission lint`, `pr lint`
Measurement	`usage`

The CLI is the reference implementation of every rule in this document. The MCP server and the Web UI are alternative front ends over the same library code, not reimplementations.

4.2 The ingestion pipeline

```
PDF ──▶ parser ──▶ sectioner ──▶ scaffolder ──▶ summarizer ──▶ builder
       (docling)   (headings)    (L3+L1+L2)     (LLM CLI)     (gates)
                        │
                        ├── images ──▶ content-addressed assets
                        └── tables ──▶ verbatim, into L2 and L3 alike
```

Parsing uses docling to recover layout, tables and images from the PDF. **Sectioning** prefers the document's own bookmarks/outline; with `--no-bookmarks` it falls back to heading patterns, buckets front matter, and derives readable slug ids for headings it cannot parse. Every decision the sectioner makes is reported — demoted headings, fallback ids, out-of-order numbering, duplicate ids, bookmarks the split missed — so the split is auditable rather than magical.

Scaffolding writes L3 immediately and complete, creates L2 with the tables already in place and the prose blank, and writes L1 with every section `pending`. At this point the document is fully usable at L3 and fully honest about what is missing.

Summarization is the only step that involves a model. It shells out to a headless LLM CLI rather than an API, so it runs on an existing subscription. It is bound by fixed rules: write in the source's own language, never rephrase identifiers or numbers, never touch tables, and prefer the original wording over invention.

Building is the gate. It refuses to let an incomplete or drifted store proceed.

4.3 The search index

`kb query` and the MCP `kb_search` tool run hybrid retrieval:

1. **Tag pre-filter at L0.** Nearly free, and it is what makes a large federation searchable at all. A document id is indexed as a synthetic tag, so a caller can scope to one document without a separate flag.
2. **Keyword leg** — SQLite FTS5 over section text.
3. **Semantic leg** — optional KNN over embeddings, available when the `embed` extra is installed.
4. **Fusion** — reciprocal rank fusion over the two legs. Each leg is capped at 50 results before fusion, and the caller is told when matches were dropped.
5. **Budgeting** — L2 content of the top hits is loaded until the token budget is reached. Sections are never cut mid-way, which is what keeps returned tables verbatim; the budget is therefore advisory and the first result is always returned whole.

No model call happens during a lookup. Retrieval is pure code: fast, free and deterministic.

The index is persistent but derived. It can be deleted and rebuilt from the Markdown at any time, which is why the Markdown, not the index, is the source of truth.

4.4 The service process

One process serves three audiences over HTTP:

Surface	Audience	Auth
`/mcp`	AI agents	Bearer token
`/api/…`	programmatic callers	Bearer token or session cookie
`/ui`	people	sign in with a hub account, session cookie stored; the token opens it read-only

The UI has a **Ticket** menu beside Document (`/ui/tickets`) for the missions and tickets that `ba` repos publish (§8.4).

The same process also runs over stdio for a local agent. In every mode a hub is **mandatory** — resolved from `--hub`, the environment, or `.kb/config.yaml`, in that order — and the server refuses to start without one. It searches only the hub's `federation/`, never its own working `.kb/`. That single rule is what makes "published" mean something.

Fourteen MCP tools are exposed: `kb_search`, `kb_get_section`, `kb_context_new`, `kb_resolve`, `kb_ticket_lint`, `kb_ticket_check`, `kb_ticket_stale`, `kb_ticket_fetch`, `kb_question_ask`, `kb_question_list`, `kb_question_answer`, `kb_mission_lint`, `kb_mission_next` and `kb_doctor_hub`. The three `kb_question_*` tools work on the ticket Q&A store, and `kb_question_ask` and `kb_question_answer` are the only MCP tools that write (see 8.5); the other twelve only read.

---

5. Federation

5.1 The hub

A hub is an ordinary Git repository with the same `.kb/` layout as any other, plus a machine-generated `federation/` directory:

```
hub/
├── .kb/                          the hub's own documents, if any
└── federation/
    ├── index.yaml                aggregate L0 across every entry
    ├── registry.yaml             optional: which repo may publish which id
    ├── <repo-a>/                 a full mirror of repo A's .kb/ (L0-L3)
    ├── <repo-b>/
    └── <mid-hub>/<repo-c>/       a nested entry from an upstream chain
```

Each participating repository mirrors its **full** store — all four layers, not a slim snapshot — so the hub can answer any query without reaching back to the source repository. There is no runtime coupling between a hub and its children.

5.2 Publishing

`kb publish` mirrors `.kb/` into `federation/<repo-id>/` and rebuilds the aggregate index. The destination comes from configuration, not from a flag, so a repository cannot accidentally publish into the wrong hub.

Mode	When it applies
Pull request (`--pr`)	The normal path. Needs a git remote and a `gh` able to open a PR on that host.
Direct commit (`--direct`)	Refused on a *governed* hub with a remote. Intended for a purely local hub.
Automatic	Direct only for a hub with no remote; otherwise a PR, or an explicit refusal.

Publishing copies **artefacts only** — documents, indexes, manifests, assets. Configuration files and dotfiles are withheld and named in a warning. This is a security property, not tidiness: the `hub:` setting routinely carries a

credentialed URL, and mirroring it would publish a token.

`kb ci-publish` is the CI route. It runs inside the publishing repository's own GitHub Actions job, authenticates to an intake service with GitHub Actions OIDC, and therefore needs no long-lived secret anywhere. `child`, `dev` and `ba` repositories all publish through it; the `ba` flow is described in §8.2.

5.3 Multi-tier federation

A hub may itself declare an upstream hub. Publishing such a repository mirrors its `federation/` — not its own `.kb/` — into the upstream, preserving the nested layout. Depth is unbounded, entry ids become paths, and a chain that would loop content back is refused with an explicit cycle error.

This gives search a natural scope control: query a team hub for the team's knowledge, query the root hub for everything. No filtering logic is required, because the topology already expresses the scope.

5.4 Integrity on the hub

- `federation/index.yaml` records a content hash per snapshot, so `kb doctor` detects content modified in place on the hub rather than through a publish.
- `kb reindex` rebuilds the aggregate index from the entries when it has drifted.
- `kb doctor` reports structural faults, manifest/file disagreements, files a publish would never have written, a hub clone storing a credential in its own git config, and a git older than the 2.31 floor.

5.5 Asset offloading

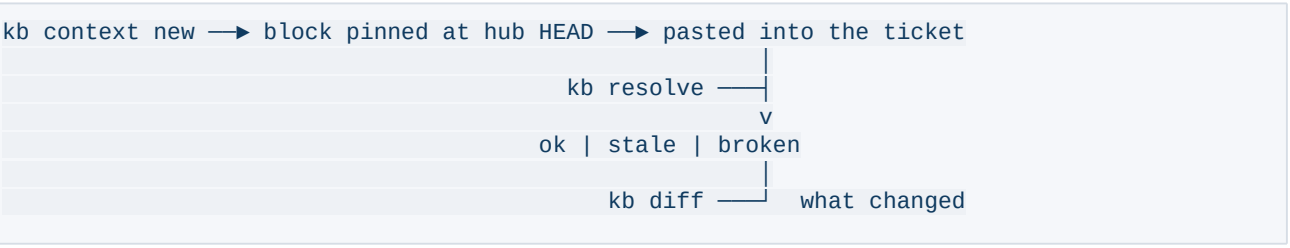
A hub may declare an S3 asset store. Publishes then upload images to the bucket instead of committing them, recording what was diverted alongside each document. The service serves diverted images from the bucket with a local disk cache. The default keeps assets in Git like any other file; the choice is per-hub and invisible to callers.

---

6. Citations that pin a version

Knowledge changes. A requirement written against last quarter's revision must not silently start meaning something else.

A `kb-context` block is a machine-readable citation embedded in a ticket, a spec or an acceptance criterion. It records the hub's commit at the moment of writing and a set of repo-qualified section references.



- **`ok`** — the pinned content still matches what is published.
- **`stale`** — the store was amended after the citation was taken. `kb diff` reports exactly what changed between the two revisions: titles, L1 summaries, the L2 slice, the L3 original, added and removed sections, and manifest reorders.
- **`broken`** — the reference no longer resolves at all.

`kb doctor --context` runs the same check in CI, so a stale requirement can be flagged before merge rather than discovered during implementation.

---

7. Code knowledge

A `dev` repository publishes two documents about itself. They are deliberately separate.

Document	Origin	Regenerated	Needs review	Tagged
`<repo_id>-code`	`kb code-ingest`, deterministic, no model	on demand from the working tree; CI publishes on every merge	no	`code, generated`
`<repo_id>-svc`	drafted by a model from that evidence, corrected by a human	never	yes	`code, curated`

They cannot share one document for three independent reasons: `-code` is overwritten wholesale while `-svc` accumulates; `-svc` carries `pending` sections during a seed while `-code` must always build clean; and `-code` may be merged without a human by the repo's auto-merge switch while `-svc` needs human review (done locally, before publish).

`-code` is never committed: its revision lives in its manifest, not in `index.yaml`, so a dev repo's committed index does not change when code does.

They are joined by **section id**: `svc.<name>` exists in both. `-code` supplies a service's alias, label and technology; `-svc` supplies its responsibility. Together they fill all four arguments of a C4 `Container(...)`, which is what lets a BA agent draw an accurate architecture diagram from published knowledge.

7.1 Extraction

`kb code-ingest` is organised by **artefact kind rather than programming language**, because Strata is adopted across many project lines and a single-stack extractor set would leave most repositories with nothing.

Extractor	Produces	Reads
`services`	`svc.<name>`	compose files, Dockerfiles, k8s manifests, solution and workspace files
`deps`	`dep.<ecosystem>`	manifests across Python, JavaScript, Java, .NET, Go, PHP, Rust, Swift and Dart
`commands`	`cmd.<purpose>`	package scripts, Makefiles, tox, shell scripts, CI workflow steps
`tree`	`struct.tree`	the files Git tracks, so ignored directories never appear
`schema`	`db.<table>`	SQL migrations, Prisma, Alembic, EF, and explicitly named SQLite files
`integrations`	`int.<name>`	environment-variable **keys** only
`api`	`api.<tag>`	in-repo OpenAPI and Swagger contracts

Output is an ordinary four-layer document, so build, query, federation and the Web UI read it with no special-casing.

7.2 Two security properties

These are properties of the design, not conveniences that could be relaxed.

1. **Keys, never values.** The only environment file the integrations extractor opens is an example/sample/template file — never a real `.env` — and its pattern has no capture group around the value at all, so there is nothing to leak regardless of what a matched line contains. Compose `environment:` blocks *are* read and do hold real inline values, but only the key survives.
2. **Explicit database input.** SQLite schema is read only from paths named on the command line. A scratch or fixture database sitting in the repository can never become published company knowledge by accident.

7.3 Determinism

Extractors are pure functions of the tracked tree plus explicitly named inputs. Sections, dependencies and tables sort deterministically, column order is preserved because it carries meaning, serialised keys sort on write, path separators and line endings normalise. Re-running on the same commit on the same platform produces byte-identical output, which is what makes the CI publish a true no-op on an unchanged tree.

Two honest limits: a handful of glob matches case-normalise per operating system, so the guarantee is per-platform rather than universal (CI is always Linux, so what is published is stable); and the manifest's revision derives from the HEAD commit while the working tree may hold uncommitted changes — detected and reported as a `dirty_tree` warning.

---

8. Plan knowledge

A `ba` repository is the third publisher. What it publishes is not domain knowledge and not code knowledge but **the plan**: the missions and tickets its analysts have reviewed, as one generated document, `<repo_id>-plan`.

Document	Origin	Regenerated	Needs review	Tagged
`<repo_id>-plan`	`kb plan-ingest`, deterministic, no model, offline	on demand from `missions/` and `tickets/`	no: a fully ticked Definition of Ready is the review	`plan, generated`

It follows `-code` in every mechanical respect. It is never committed (`.kb/*-plan/` is gitignored, and so is the `ba` repo's generated `.kb/index.yaml`), its revision lives in its manifest, and it is an ordinary four-layer document that build, query, federation and the Web UI read with no special-casing. Its sections are written `summarized`, never `pending`, so `kb build` passes without a model.

8.1 What qualifies, and the document's shape

A mission is published when it passes the structural `kb mission lint` **and** every box of its `## Definition of Ready` is ticked. A ticket needs the same from `kb ticket lint`, and `kb mission next` must not demote it to `draft` because its parent mission moved on. Citation resolution is **not** a gate: it needs the hub, and the pull-request lint already enforces it. Each item left out is reported as `skipped: <file> — <reason>` and never fails the command, so an unready item simply is not published, and one that stops qualifying drops out at the next publish.

The document has one group per mission and a `tickets` group for the rest. Sections are `## <id> <title>`, with `M-<slug>-US<n>` as a story's id. L3 is the source with the title dropped, headings demoted one level and the BA-internal `## Review record` and `## Definition of Ready` removed; L2 keeps every table byte for byte (so the table invariant holds without relaxing `kb build`) plus the prose that summarises the item. A line starting with `##` inside a code fence would split a section for every reader, so such an item is skipped until the line is indented.

8.2 The publish flow

Publishing is on demand. The hub's intake accepts only a `kb-publish/*` tag (never a branch push), so the plan travels the same tag-driven route a `child` repo uses:

```
 BA runs kb publish
      |  kb plan-ingest + kb build      (local preview = what CI will send)
      |  tag kb-publish/<ts>, push the tag
      v
kb-publish.yml (tag trigger, OIDC)
      |  kb plan-ingest + kb build + kb ci-publish
      v
intake: verifies the OIDC token, looks the repo up in
        federation/registry.yaml  -->  PR on the hub
      |  review + merge
      v
federation/<repo_id>/<repo_id>-plan  -->  kb query / MCP / Web UI
```

The registry entry, never the payload, decides the repo id it may write. (Its optional `workflow:` pin is compared with the full `job_workflow_ref`, which includes the tag, so it stays unset for tag-triggered publishing.) Nothing publishes on merge: the analyst decides when, and `/kb-publish` shows what changed before asking. A repo with no `intake:` configured falls back to the snapshot route.

8.3 Drift: keeping missions and tickets current

A mission is written once and reality then moves. Each mission and ticket carries a `> Revision: <N>` and an append-only `## Change log`, and two checks detect drift. Both run in the daily `kb-doctor` workflow through `kb ticket stale`, which keeps one `kb-stale-citations` issue:

```
 KB-context drift       cited hub section at its pinned hub commit
                       vs the hub worktree        -->  ok | stale | broken

Architecture drift     ids cited in Services & order / Technical grounding
                       (svc.* db.* api.* int.* cmd.*, minus [NEW: D<n>])
                            |
     Grounded on: <repo>-code @ <rev>   (the dev repo's commit)
                           |  walk the hub history of the -code manifest;
                           |  newest hub commit whose revision == <rev>
                           v
     <repo>-code §<id> @ <that hub commit>  --> the same resolve step
                           -->  stale | broken, kind "architecture"
     no hub commit carries <rev>  -->  one finding, "baseline not on hub"
```

Drift is judged **per cited section**, never on the document revision: `-code`'s revision moves on every dev commit, so comparing it would flag every mission on every merge. A merge that touches nothing a mission cites produces no finding.

`kb ticket check` follows the same rule. A moved `Grounded on:` revision is never a failure by itself: the check finds the baseline the same way, then lets the ticket's own fact checks against the current `-code` decide. Every cited fact holds → PASS, with a `[note]` when the cited sections are unchanged or a `[warn]` plus a diff for each one that changed. A cited fact is gone → FAIL with `re-run /sa-ticket-ground`. Decision errors never carry that line.

An amendment (`/ba-mission-plan --amend`, `/ba-ticket-author --amend`) records its trigger (`kb-context`, `architecture`, `scope`, `delivery`), bumps the revision and appends a change-log row. A ticket amend runs `kb ticket check` every time and invokes the SA for an `architecture` trigger, or when the check fails with `re-run /sa-ticket-ground`; a mission amend invokes the SA and the check only for `architecture`. Delivered stories are never rewritten: change lands as new stories or as superseding decisions (`SUPERSEDED`), and a ticket pinned `> Parent mission: M-<slug> @ rev <N>` is reported `draft` by `kb mission next` when its mission has since changed that story.

8.4 Status on the hub

Story status is derived, read-only, from what the hub already holds: `done` when the id is in a `hist.*` row of the dev repo's `-svc`, `ready` when the ticket is published, `not ready` when it is not, `blocked` with reasons. `kb mission next --from-hub` computes it in the CLI; the Web UI's **Ticket** menu renders it, and `*-plan` documents appear under Ticket, not Document, and the overview counts per kind. Nothing is stored and there is no write path. In the other direction, a dev given only a ticket id runs `kb ticket fetch`, which reads the ticket from the hub's `-plan` and restores its source shape.

8.5 Ticket questions

A Dev who meets a business question while designing or planning a published ticket asks the BA through `kb_question_ask`; the question is stored in SQLite at `<hub root>/.kb-qa/qa.db`, beside `.kb-auth/` and out of git. This is the one thing the hub stores that is not in git, so the three question tools work only against the hub itself, that is the shared hub's HTTP server or the stdio server run inside the hub's own checkout (`--kb <hub>/.kb`); any other target is refused: a dev repo's stdio server, or a hub URL read through a local cache clone, answers `Q&A needs the shared hub server — point MCP at it (kb mcp-setup)`, because a store there would not be the shared one.

A question's state is derived, never stored. It is *open* until answered. An answer that changes the ticket records the revision that carries it (`answer_rev`) and the changed sections; the question is *waiting for publish* until the hub's copy of the ticket (its own `> Revision: N`, not the hub's git HEAD) reaches that revision, then *answered*. The Web UI's **Q&A** menu renders it read-only. The BA answers with `/ba-answer-questions`, which amends the ticket (change-log Trigger `question`), leaves the Definition of Ready to be re-ticked, records the answer and never commits, pushes or publishes; the Dev side raises business questions from `dev-design` and `dev-plan` and sends technical ones to `dev-sa`.

---

9. The trust model

Strata's correctness argument is a chain of four links, each of which can fail loudly and none of which can be skipped.

Link	Enforced by	Fails how
The original is preserved unmodified	L3 written by code at ingestion	A changed L3 invalidates every summary written against it
Tables survive summarization intact	`kb build` bidirectional table diff	Non-zero exit, the change cannot merge
Prose is constrained, not free-form	Fixed summarization rules plus quality checks	Warnings by default, errors under `--strict`
A human agrees before anyone can read it	The pull request on the hub	Content is unreachable through query, MCP and the UI until it merges

Note what is *not* claimed. The model is not trusted, and the design does not require it to be: it is confined to prose, its output is bounded by automated checks, and it cannot publish. Equally, the hub registry is a **mistake guard**, not an authentication mechanism — a publisher's remote URL is self-asserted. Branch protection on the hub and the OIDC intake are the controls that actually hold. The same holds for a `ba` repository's plan: what ships is chosen by a human's ticked Definition of Ready, on a tag the human pushes, and it still reaches readers only through a hub pull request.

9.1 Service security

- Every response carries a content security policy, frame denial, nosniff, referrer and permissions policies; HSTS over https.
- The UI session cookie is a signed, expiring value keyed with `<hub>/.kb-auth/session.key` — never the bearer token, which every child repo holds. It carries the username, not the role; the role is read from the account database on every request. Accounts live in `.kb-auth/`, which is never committed.
- The Registry page (`/ui/registry`) is admin-only, so the bearer token can view the UI but never change the registry. Each edit is a commit on the hub's default branch, pushed with the intake GitHub App, never force-pushed. A repo's `auto_merge` switch makes intake merge its whole publish PR right after opening it; a failed merge leaves the PR open and the publish `done` with `merged: false`.
- The ticket Q&A store (`<hub>/.kb-qa/qa.db`, never committed) is written only by the three `kb_question_*` MCP tools, and only on the hub itself (not a cache clone). Any caller the hub authenticates may write; the names they give are self-declared, so they track who said what and authenticate nobody. At most 50 questions per ticket can be open.
- A failed bearer-token attempt shares the login form's rate-limit bucket, so the lockout cannot be side-stepped through the header.
- Behind a reverse proxy, the trusted-proxy count **must** be configured. The default ignores forwarded headers and keys on the socket peer; left unset behind a proxy, every caller shares the proxy's address and one caller can exhaust the limiters for everyone.
- HTTP authentication is a bearer token for `/api`, `/mcp` and CI plus local hub accounts for the UI — no OAuth or SSO. That is adequate for an internal network and is not ready for the public internet.

---

10. Deployment topologies

10.1 Single team

```
authors ──▶ child repo ──▶ hub repo ──▶ one service process
                                          (MCP + REST + UI)
```

One hub, one or more children, one service. The hub may be the same repository that authors content. This is the starting configuration and it scales further than most teams expect, because the service is stateless and the store is a Git clone.

10.2 Organisation-wide

```
   domain teams          product teams         requirements
  child × N             dev × N               ba × N
      \                    |                    /
```

```
        \                  v                    /
         +-----------▶  root hub  ◀-------------+
                           |
                   shared service process
```

Every repository publishes to one hub; every consumer reads from it. The `ba` and `dev` repositories contribute no domain content: `dev` repositories publish only knowledge about themselves, and `ba` repositories only their plan.

10.3 Federated organisation

```
  team hub A ─┐
team hub B ──┼──▶ root hub
team hub C ─┘

query a team hub  → that team's knowledge
query the root hub → everything
```

Intermediate hubs give each division its own searchable scope while still rolling up to a single organisational view. Scope is expressed by topology rather than by query-time filtering.

---

11. Operational characteristics

Property	Value
Runtime state	None. Every artefact is a file under version control.
Query cost	No model call. Local index read plus file reads.
Ingestion cost	One-off per document; the first run downloads a layout model (~500 MB) and caches it.
Summarization cost	One model call per section, once, re-runnable on demand.
Scaling limit	Practically, the size of a Git clone and an FTS5 index — tens of thousands of sections are unremarkable.
Failure mode	Degraded, not silent: a broken store fails a build; a stale citation reports `stale`; an unavailable semantic leg warns and falls back to keyword search.
Platforms	Python 3.11–3.13, Linux and Windows, gated on every release.
Recovery	Delete the index and rebuild it; re-clone the hub; re-publish from the source repository.

---

12. Extension points

Strata is intended to be extended along four seams, each of which was designed to absorb change without touching the core.

1. **Sectioning strategy.** Bookmark-driven and pattern-driven splitting are two implementations of one interface. A document family with its own structure gets a third rather than a special case in the pipeline.
2. **Extractors.** `kb code-ingest` dispatches to independent extractors keyed by artefact kind. A new ecosystem is a new extractor and a new section prefix, not a change to the document format.
3. **Retrieval legs.** The keyword and semantic legs are fused generically. A third ranking signal joins the fusion rather than replacing the pipeline.
4. **Section-id prefixes.** The prefix table is a published contract that agent tooling keys on. New knowledge kinds claim new prefixes; existing ones never change meaning.

---

13. Summary

Strata's architecture is a single idea applied consistently: **put the expensive, error-prone work where it can be checked.**

Sectioning is mechanical, so it can be reported and audited. Tables are mechanical, so they can be diffed. Prose is generated, so it is bounded by rules and by quality checks. Publication is human, so it is a pull request. Retrieval is mechanical, so it is free and repeatable.

Nothing in the system asks anyone — a person or a model — to be trusted where a check could be run instead.